Лабораторная работа № 2: Поиск экстремума функции методами хорд, секущих и Ньютона

Индивидуальные задания

Построить в MS Excel график функции в соответствии с вариантом. Найти экстремум функции, используя изучаемые методы, в соответствии с предлагаемой методикой.

Таблица 1

Вариант	Уравнение ***У*= …**
1	$2x^4 – 8x^3 + 8x^2 – 1$
2	$x^4 – 4x^3 - 8x^2 + 1$
3	$2x^3 - 9x^2 – 60x + 1$
4	$x^2 – \sin x$
5	$x^2 + \sin x$
6	$x^4 – \sin x$
7	$x^3/3 – \cos x$
8	$3x^4 + 8x^3 + 6x^2 - 10$
9	$3x^4 + 4x^3 - 12x^2 - 5$
10	$3x^4 + 4x^3 - 12x^2 + 1$
11	$x^4 – x^3 - 2x^2 + 3x - 3$
12	$x^4 + x^3 - 10x^2 – 34x - 25$
13	$2x^4 – 7x^3 + 8x^2 - 2$
14	$x^4 – 4x^3 - 9x^2 + 3$
15	$2x^3 - 9x^2 – 37x - 1$
16	$2x^2 - \sin 5x$
17	$x^3/2 + \cos x$
18	$2x^4 - 3\sin 3x$
19	$2x - 4\sin x$
20	$2x^4 + 8x^3 + 6x^2 - 11$
21	$x^4 + 4x^3 - 11x^2 - 5$
22	$3x^4 + 5x^3 - 12x^2 + 4$
23	$x^4 – x^3 - 2x^2 + 3x - 5$
24	$x^4 + x^3 - 10x^2 – 31x - 22$
25	$x^4 – 2x^3 - x^2 + 3x -4$

Методика выполнения заданий

Метод хорд

В основе метода лежит линейная интерполяция функции по двум значениям, имеющим противоположные знаки. Метод хорд дает решение задачи для достаточно малых ε за меньшее число арифметических операций, чем метод половинного деления.

Пусть нужно найти корень уравнения ***f(x) = 0*** на отрезке *[a,b]*, причем известно, что ***f(x)*** непрерывна на *[a,b]* и *f(a)*f(b)* < 0. Кроме того, пусть ***f'(x)*** и ***f"(x)*** на отрезке *[a,b]* сохраняют свой знак. Заменим функцию ***f(x)*** на отрезке *[a,b]* линейной функцией (рис.1), составив уравнение прямой, которая проходит через точки *(a, f(a))* и *(b, f(b)):*

$$\frac{y - f(a)}{f(b) - f(a)} = \frac{x - a}{b - a} \ ;$$

$$y = f(a) + \frac{x - a}{b - a}[f(b) - f(a)] \ .$$

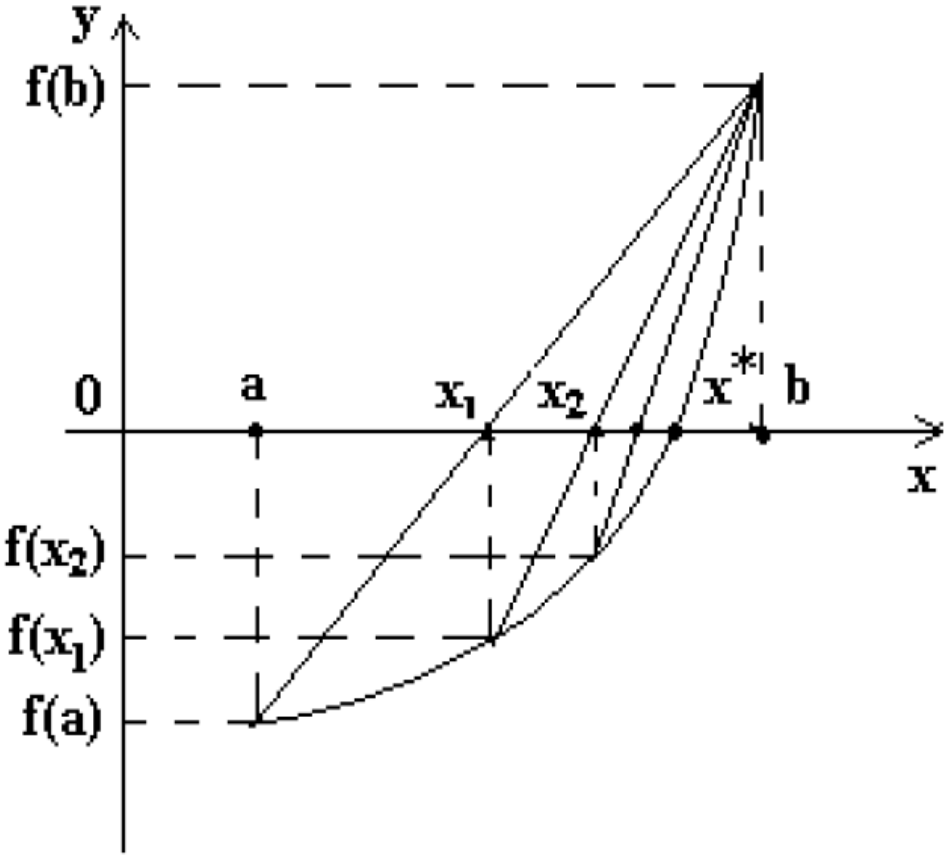

Рис. 1

Линейная функция $P(x) = f(a) + \frac{x-a}{b-a}[f(b) - f(a)]$ на концах отрезка *[a,b]* принимает те же самые значения, что и функция ***f(x)***. В качестве приближенного корня уравнения ***f(x)*** = 0 возьмем точный корень уравнения *P(x)=0*. Это значение x_1 (первое приближение) определяется из соотношения

$$f(a) + \frac{x_1 - a}{b-a}[f(b) - f(a)] = 0 ,$$

откуда следует, что

$$x_1 = a - f(a)\frac{b-a}{f(b)-f(a)} .$$

Далее рассмотрим отрезки $[a,x_1]$, $[x_1,b]$ и выберем из них тот, на концах которого функция *f(x)* имеет значения противоположных знаков. Те же вычисления выполним на выбранном отрезке и получим второе приближение к корню x_2 и так до тех пор, пока не получим корень уравнения с заданной степенью точности.

Алгоритм метода следующий. До начала итерационного процесса задаем точность ε, с которой нужно получить решение, и отрезок *[a,b]*, содержащий корень. Затем:

1. Вычисляем приближение к корню: $x = a - f(a)\frac{b-a}{f(b)-f(a)}$.
2. Проверяем выполнение неравенства $|f(x)|<\varepsilon$ и, если оно выполняется, то x считаем решением, если же не выполняется, продолжаем вычисления.
3. Проверяем условие $f(x)*f(a) < 0$, и, если оно выполняется, полагаем $b=x$, в противном случае $a=x$ и повторяем вычисления с п.1.

Блок-схема алгоритма представлена на рис. 2.

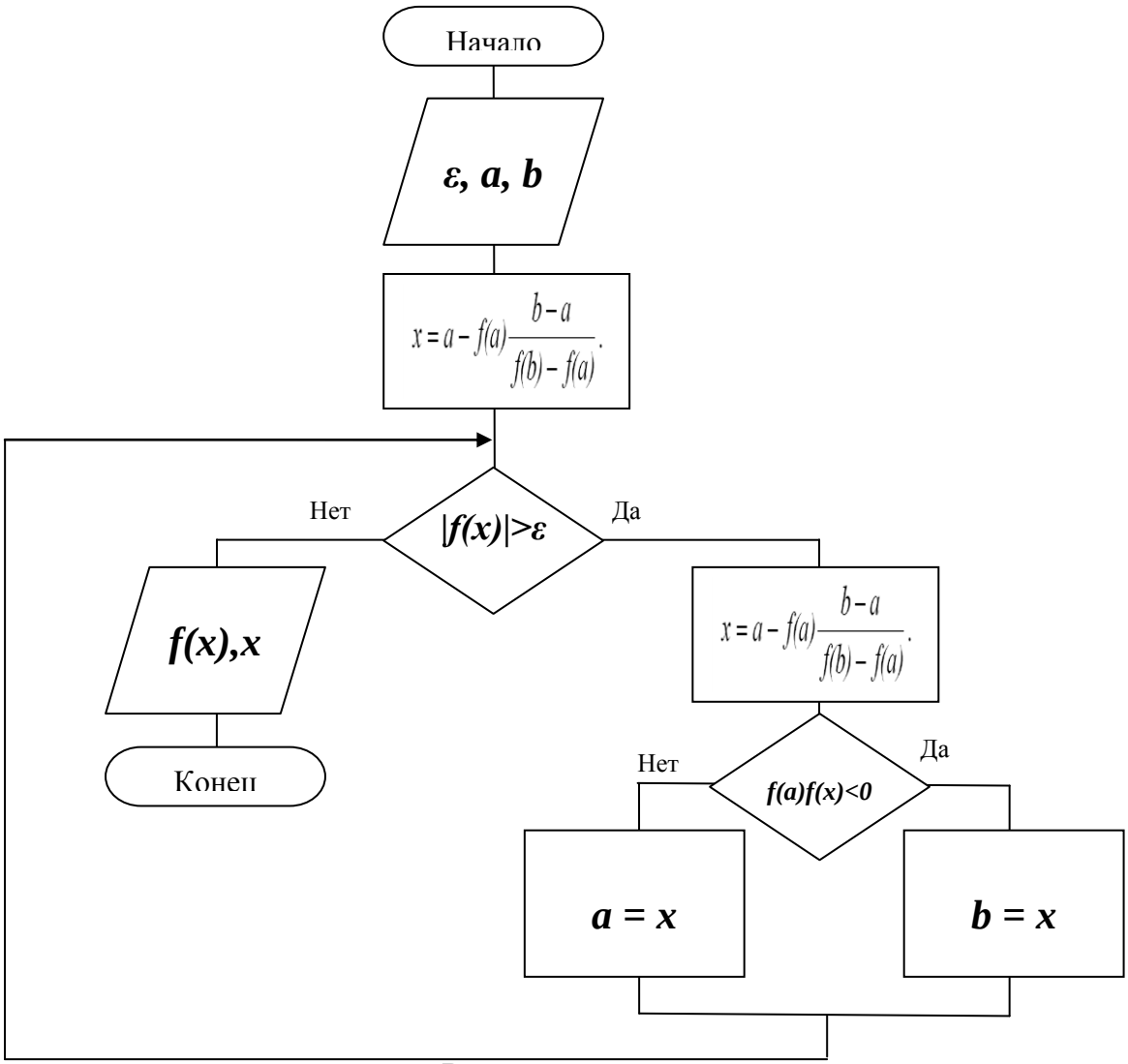

Рис. 2

Пример. Найти экстремум функции $y=x^2/2 - \sin x$.

Для решения воспользуемся VBA (Alt+F11). На рис. 3 представлена созданная пользовательская форма.

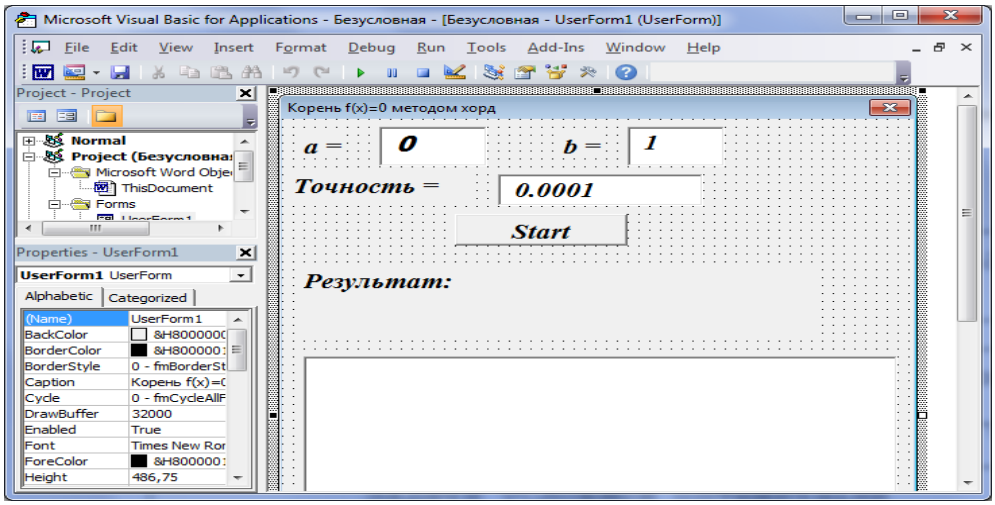

Рис. 3

Программный код для поиска экстремума функции $F=x^2/2 - \sin(x)$ (поиска корня уравнения $f' = x - \cos(x) = 0$):

```
Private Sub CommandButton1_Click()
a = Val(TextBox1.Text): b = Val(TextBox2.Text)
toch = Val(TextBox3.Text)
x = a - f(a) * (b - a) / (f(b) - f(a))
TextBox4.Text = TextBox4.Text + "f(a) = " + Str(f(a)) + "  f(b) = " + Str(f(b))
+ Chr(13)
TextBox4.Text = TextBox4.Text + "x = " + Str(x) + "  f(x) = " + Str(f(x)) +
Chr(13)
Do While Abs(f(x)) > toch
    k = k + 1
    x = a - f(a) * (b - a) / (f(b) - f(a))
    TextBox4.Text = TextBox4.Text + "f(a)=f(" + Str(a) + ")=" + Str(f(a)) + "
f(b) = f(" + Str(b) + ")=" + Str(f(b)) + Chr(13)
    TextBox4.Text = TextBox4.Text + "x = " + Str(x) + "  f(x) = " + Str(f(x)) +
Chr(13)
    If f(a) * f(x) < 0 Then
                        b = x
                  Else: a = x
    End If
Loop
Label4.Caption = Label4.Caption + "Y = " + Str(f(x)) + "  x = " + Str(x) + "
k = " + Str(k)
End Sub
Function f(n) As Single
f = n - Cos(n)
End Function
```

Результат запуска пользовательской формы представлен на рис. 4.

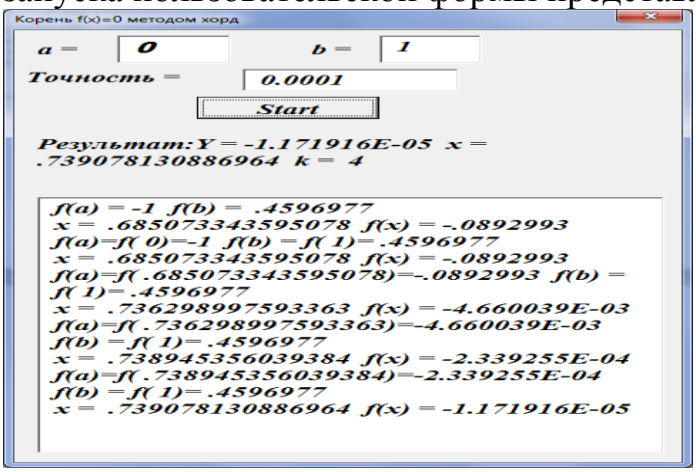

Рис. 4

Метод секущих

Метод реализуется алгоритмом метода хорд, только a и b взяты с одной стороны от корня и не фиксируются. Геометрическая интерпретация метода состоит в следующем (рис. 5). Через точки a_0 , b_0 проводим прямую (секущую) до пересечения с осью Ох. Получаем точку $\boldsymbol{x_2}$ и из нее восстанавливаем перпендикуляр к оси Ох до пересечения с графиком функции $y=f(x)$. Получим точку $\boldsymbol{b_1}$. Через точки $\boldsymbol{a_1 = b_0}$ и $\boldsymbol{b_1}$ проводим секущую - получим точку $\boldsymbol{x_3}$ (пересечение секущей с осью Ох) и т. д.

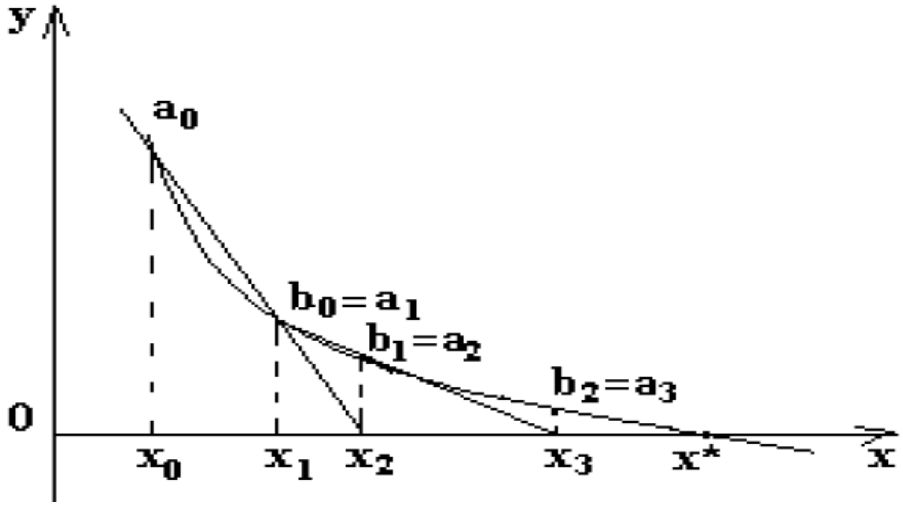

Рис. 5

В методе Ньютона требуется вычисление производных функции, что не всегда удобно. Если заменить производную секущей (рис. 6), то формула для уточнения значения корня легко выводится из подобия прямоугольных треугольников BCD и Dx_1x_2:

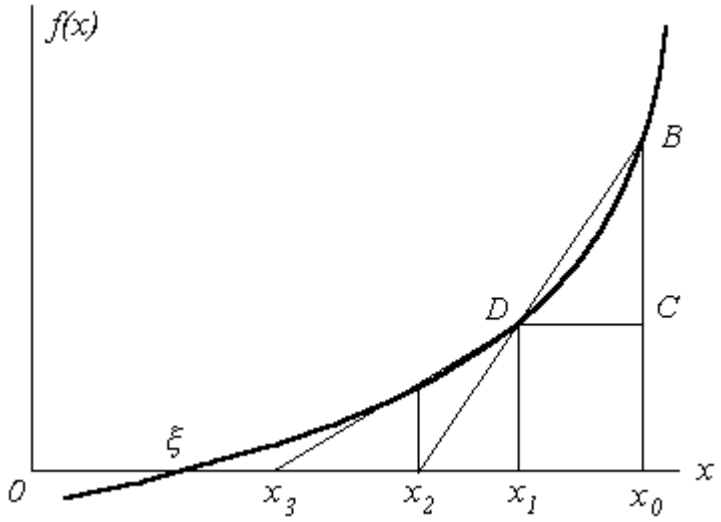

Рис. 6

$$\frac{BC}{CD} = \frac{Dx_1}{x_1 - x_2} \quad \text{или} \quad \frac{f(x_0) - f(x_1)}{x_0 - x_1} = \frac{f(x_1)}{x_1 - x_2}.$$

Тогда

$$x_1 - x_2 = \frac{(x_0 - x_1) f(x_1)}{f(x_0) - f(x_1)} \quad \text{или} \quad x_2 = x_1 - \frac{(x_0 - x_1) f(x_1)}{f(x_0) - f(x_1)}.$$

Для произвольного шага *n*

$$x_{n+1} = x_n - \frac{(x_{n-1} - x_n) f(x_n)}{f(x_{n-1}) - f(x_n)} = x_n - \frac{(x_n - x_{n-1}) f(x_n)}{f(x_n) - f(x_{n-1})}.$$

Для начала процесса требуется задать две точки x_0 и x_1 Такие процессы, где для вычисления очередного приближения требуется знать два предыдущих, называют двухшаговыми.

Блок-схема алгоритма представлена на рис. 7.

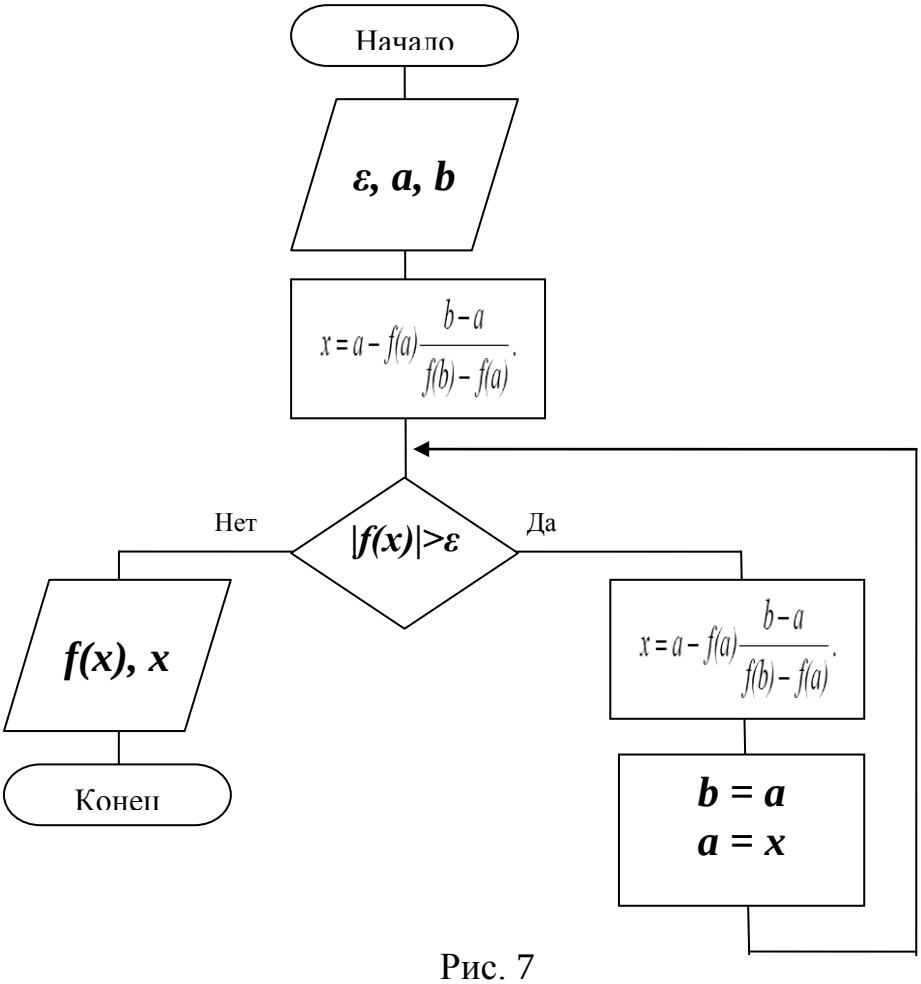

Рис. 7

Пример . Найти экстремум функции $y=x^2/2 - \sin x$.

Для решения воспользуемся VBA (Alt+F11). На рис. 8 представлена созданная пользовательская форма.

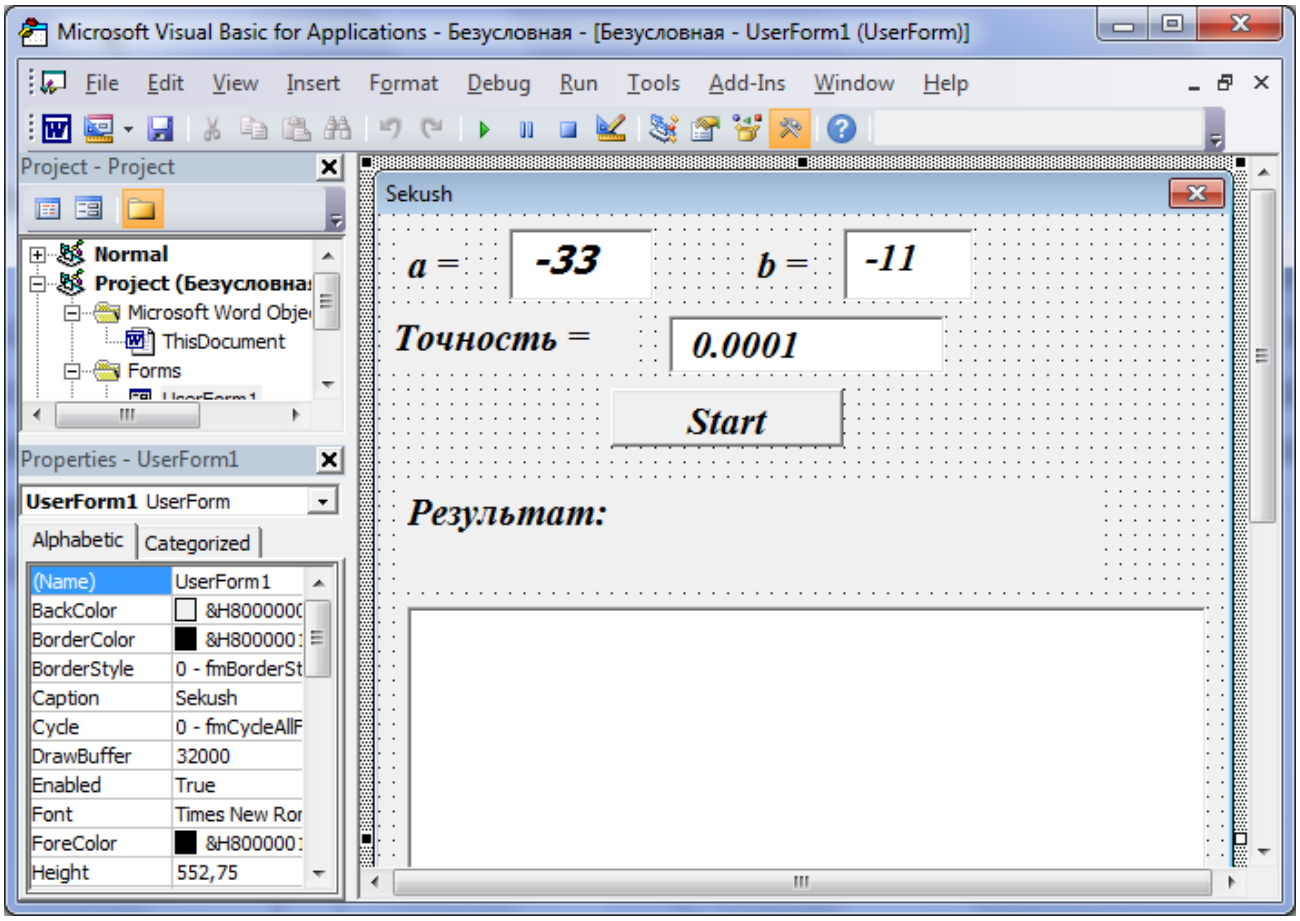

Рис. 8

Программный код:

```
Private Sub CommandButton1_Click()
a = Val(TextBox1.Text): b = Val(TextBox2.Text)
toch = Val(TextBox3.Text)
x = a - f(a) * (b - a) / (f(b) - f(a))
TextBox4.Text = TextBox4.Text + "f(a) = " + Str(f(a)) + "  f(b) = " +
Str(f(b)) + Chr(13)
TextBox4.Text = TextBox4.Text + "x = " + Str(x) + "  f(x) = " +
Str(f(x)) + Chr(13)
Do While Abs(f(x)) > toch
    k = k + 1
    x = a - f(a) * (b - a) / (f(b) - f(a))
    TextBox4.Text = TextBox4.Text + "f(a)=f(" + Str(a) + ")=" +
Str(f(a)) + "  f(b) = f(" + Str(b) + ")=" + Str(f(b)) + Chr(13)
```

```
        TextBox4.Text = TextBox4.Text + "x = " + Str(x) + "  f(x) = " +
Str(f(x)) + Chr(13)
        b = a
        a = x
    Loop
    Label4.Caption = Label4.Caption + "Y = " + Str(f(x)) + "  x = " +
Str(x) + "  k = " + Str(k)
    End Sub
    Function f(n) As Single
    f = n - Cos(n)
    End Function
```

Результат запуска пользовательской формы представлен на рис. 9.

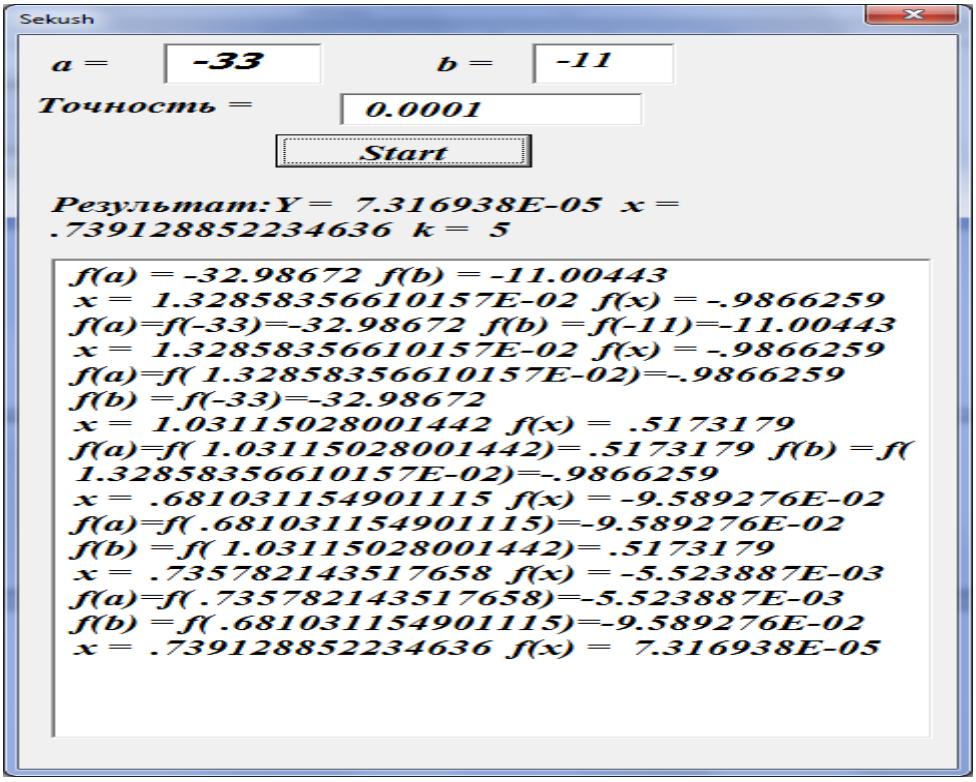

Рис. 9

Метод Ньютона

Решить уравнение $f'(x)$ =0 не всегда просто. Поэтому часто используются численные методы. Приблизительный эскиз кривой $y=f'(x)$ позволит получить приближённое решение. Если можно найти два значения a и b, таких, что $f'(a)$ и $f'(b)$ имеют противоположные знаки, то в силу предположения о непрерывности функции будет существовать корень n уравнения, причём $a<n<b$ (рис. 10).

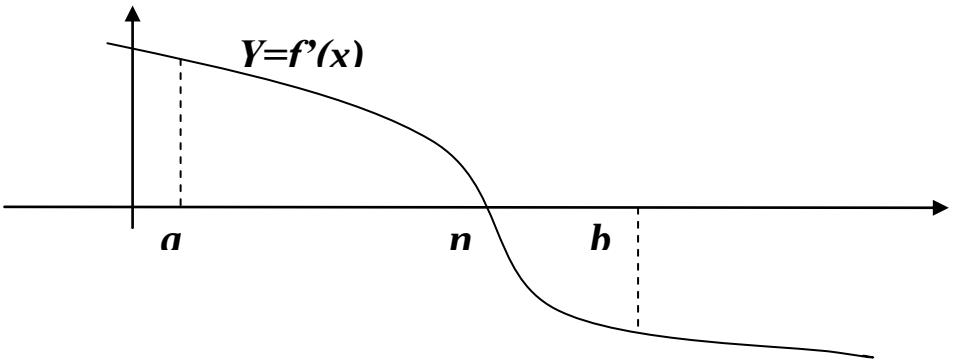

Рис. 10

Метод Ньютона позволяет улучшить относительно грубую аппроксимацию, чтобы получить корень уравнения $f'(x)$ =0. На рис. 11 точка x_0, являющаяся координатой x точки P, представляет собой аппроксимацию корня уравнения $f'(x)$ =0. Пусть PT – касательная к кривой в точке P, а T – точка, в которой касательная пересекает ось x. Тогда в общем случае OT – лучшая аппроксимация корня, лежащего в точке K.

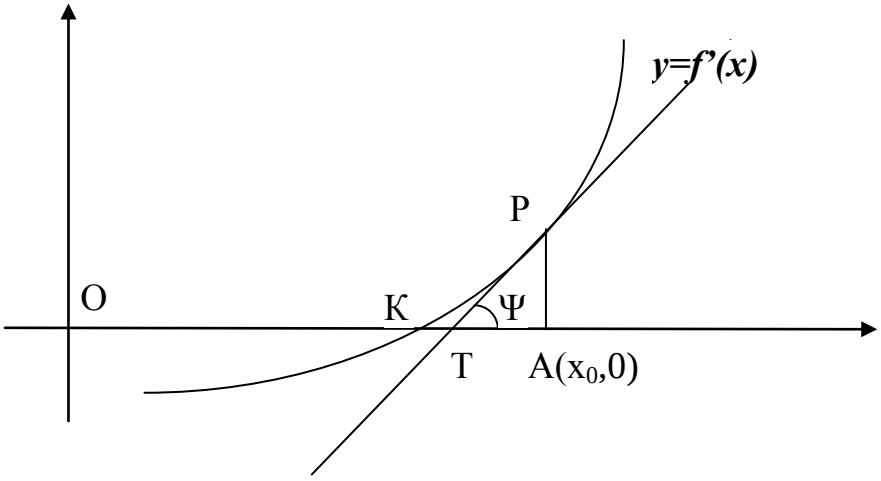

Рис. 11

$OT = OA - TA = x_0 - TA$. $PA/TA = tg\ \Psi = f''(x_0)$,
следовательно $TA = PA / f''(x_0)$ и $x_1 = x_0 - f'(x_0)/f''(x_0)$.
Аналогично получение улучшенного решения для x_2:

$x_2 = x_1 - f'(x_1)/f''(x_1)$

и в общем случае $x_{r+1} = x_r - f'(x_r)/f''(x_r)$.

Итерации можно продолжать пока не будет достигнута заданная точность ($|x_{r+1} - x_r| < e$).

Блок-схема алгоритма представлена на рис. 12.

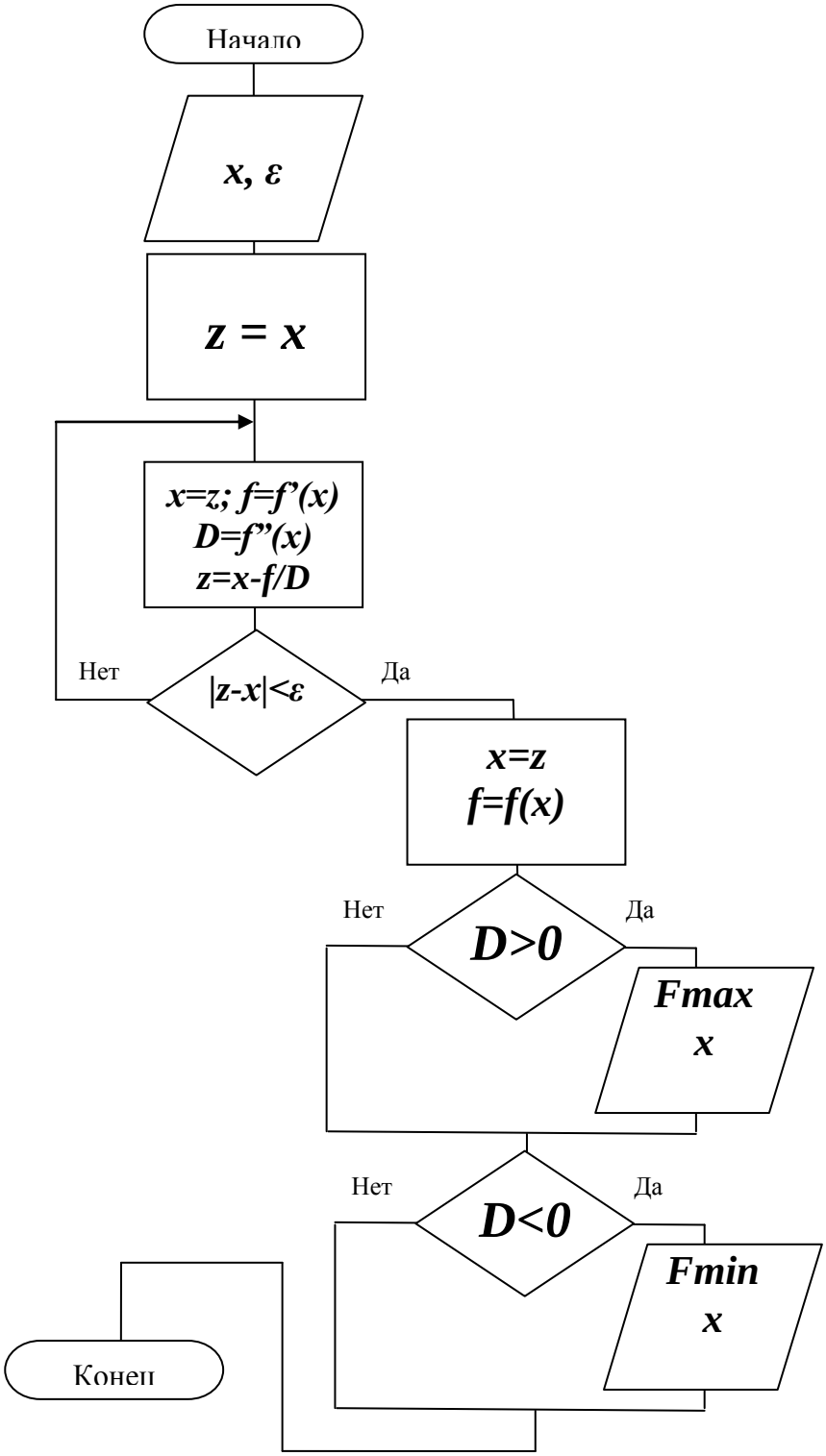

Рис. 12

Пример. Найти экстремум функции $y=x^2/2 - \sin x$.

Для решения воспользуемся VBA (Alt+F11). На рис. 13 представлена созданная пользовательская форма.

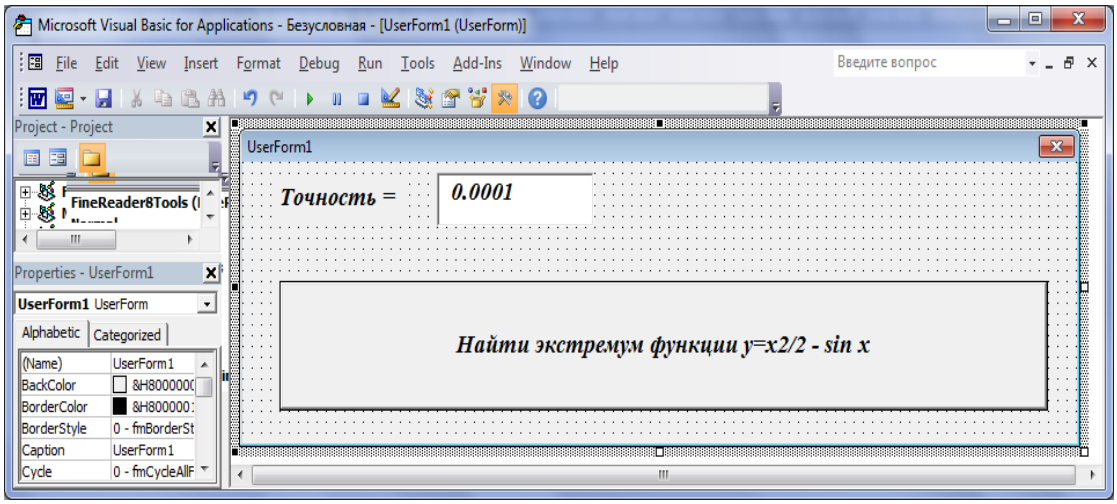

Рис. 13

Программный код

```
Private Sub CommandButton1_Click()
toch = Val(TextBox1.Text)
Z = Val(InputBox("Начальное значение x", "x = ", "0.1"))
Do
x = Z
F = x - Cos(x) 'первая производная
D = 1 + Sin(x) 'вторая производная
Z = x - F / D
Loop Until Abs(Z - x) < toch
x = Z
ff = x * x / 2 - Sin(x) 'расчёт значения функции
If D>0 Then CommandButton1.Caption="min=" & ff & "при x=" & x
If D<0 Then CommandButton1.Caption="max=" & ff & "при x=" & x
End Sub
```

Результат запуска пользовательской формы представлен на рис. 14-16.

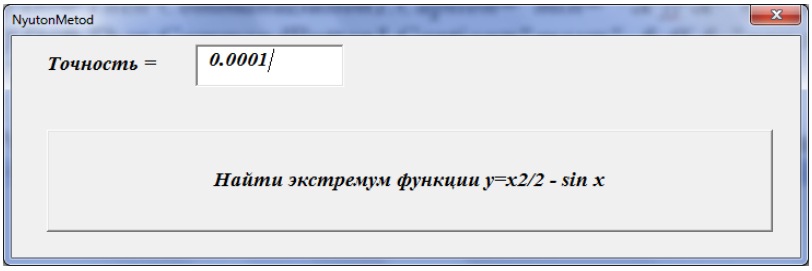

Рис. 14

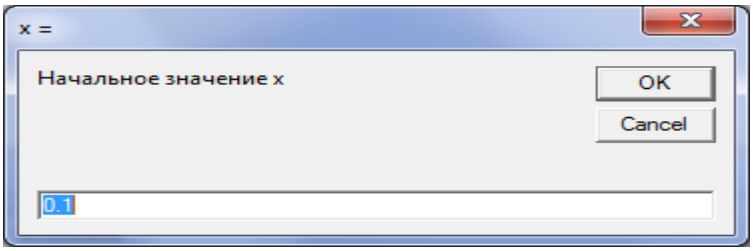
x =
Начальное значение x
OK
Cancel
0.1

Рис. 15

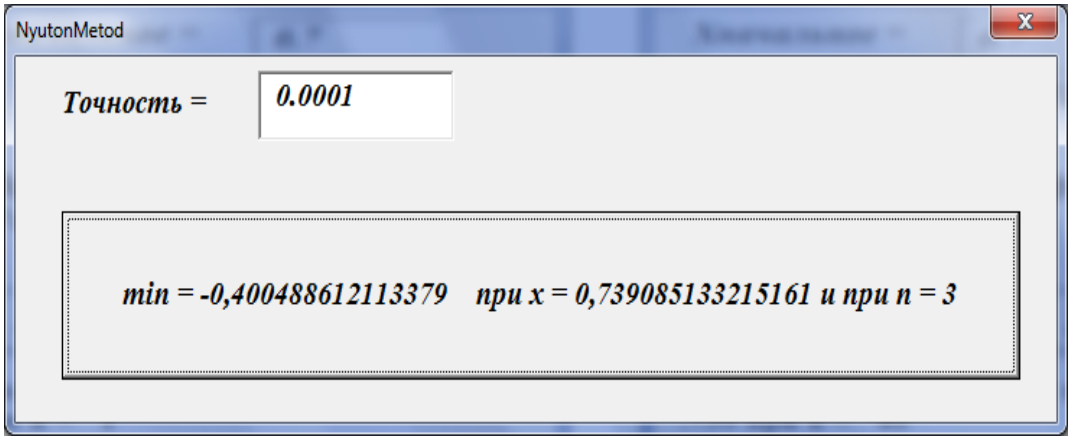
NyutonMetod
Точность =
0.0001
min = -0,400488612113379 при x = 0,739085133215161 и при n = 3

Рис. 16